

Automater 아키텍처 확장 문제집

5개의 고난이도 복합 문제 — 정적 UML / 동적 UML / ERD 포함

이 문제집은 Automater 시스템의 factory/automator/ERD 계층을 깊이 이해한 개발자를 위한 확장 시나리오입니다. 각 문제는 정적 UML(현재 구조 + 변경 지점), 동적 UML(실행 흐름 변화), ERD(스키마 영향)를 모두 포함합니다.

대상 독자: UML 해석 가능, TDD 기반 풀스택 경험이 있는 PHP/Python 개발자

- ▶ 문제 1. VideoBlock 확장 — 업로드 후 트랜스코딩 대기가 필요한 새 블록 타입
- ▶ 문제 2. WordPress BlogEditor — 두 번째 플랫폼의 구체 에디터 구현
- ▶ 문제 3. 실패한 BatchItem 선택적 재시도 — Execution Zone ERD 확장
- ▶ 문제 4. 캠페인 A/B 레이아웃 테스트 — Campaign → Layout 1:N 전환
- ▶ 문제 5. 계정 간 일일 포스팅 할당량 — Dispatcher의 비즈니스 규칙 확장

문제 1. VideoBlock 확장

상황

운영자가 블로그 포스트에 동영상을 삽입하고 싶습니다. 네이버 블로그의 SmartEditor는 동영상 업로드 후 트랜스코딩이 완료될 때까지 최대 30초를 기다려야 합니다. 운영자는 LayoutSlot에서 동영상을 media_id로 참조하고, 대체 텍스트(alt)와 자동재생autoplay)을 설정하려 합니다.

현재 정적 구조 (변경 지점 표시)

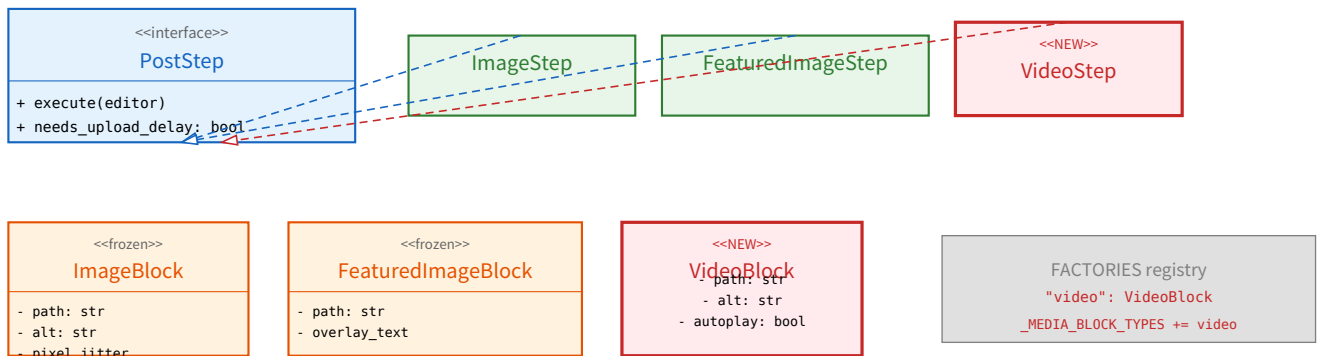


Fig 1-1. PostStep/Block 계층 — 빨간색이 추가/수정 지점

동적 흐름 (VideoStep 실행 시퀀스)

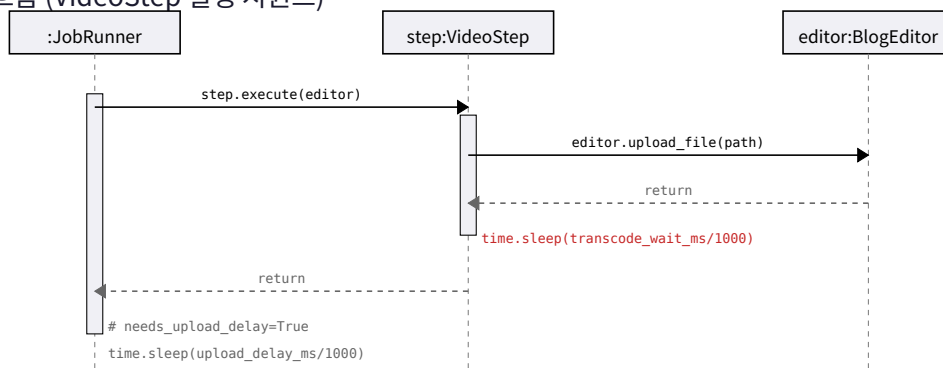


Fig 1-2. VideoStep.execute() 시퀀스 — 빨간색이 트랜스코딩 대기

ERD 영향

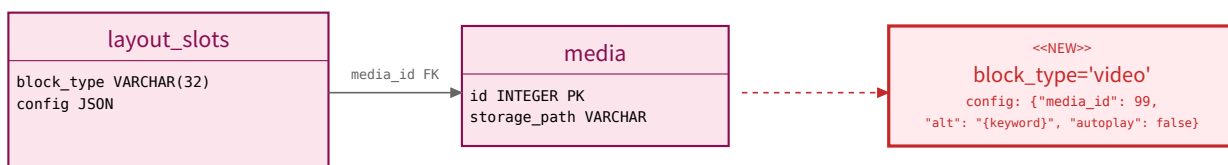


Fig 1-3. ERD — 새 테이블 없음, layout_slots.config에 새 block_type만 추가

ERD 변경 없음. VideoBlock은 기존 layout_slots 테이블의 block_type='video' + config JSON으로 표현됩니다. media 테이블도 동영상 파일(.mp4)을 기존 구조로 저장합니다.

질문

- (a) VideoBlock frozen dataclass를 options.py에 정의하시오. ImageBlock과 필드가 겹치는데 공통 부모 MediaBlock을 만들지 않는 이유를 Martin의 OCP 관점에서 설명하시오.
- (b) BlogEditor ABC에 upload_video() 같은 새 primitive를 추가해야 하는가? editor.py의 docstring에 명시된 설계 철학과 연결하여 판단하시오.
- (c) "트랜스코딩 30초 대기"를 VideoStep.execute() 안에서 처리하는 것과 BlogEditor의 새 primitive 안에서 처리하는 것의 SRP 관점 트레이드오프를 분석하시오.
- (d) VideoHandler.to_steps()에서 media_id 해석이 필요한가? 현재 ImageHandler가 media_id를 직접 해석하지 않는 이유를 코드 흐름으로 추적하시오.
- (e) test_job_builder.py 스타일로 LayoutSlot(block_type='video', config={"media_id":99, "alt":"{keyword} 소개 영상"}) → VideoBlock 변환을 검증하는 테스트를 작성하시오.
- (f) 이 변경이 기존 테스트에 미치는 영향을 분석하시오. _RecordingEditor, test_all_block_types_registered(), validate_sections() 각각에 대해 수정 필요 여부를 판단하시오.

해설

(a) VideoBlock 정의

```
@dataclass(frozen=True)
class VideoBlock:
    path: str = ""
    alt: str = ""
    autoplay: bool = False
    transcode_wait_ms: int = 30000
    filename_keyword: str = ""
```

공통 부모를 만들지 않는 이유: ImageBlock에는 saturation_jitter/exif_gps_lat이, FeaturedImageBlock에는 overlay_text/overlay_color가, VideoBlock에는 autoplay/transcode_wait_ms가 있습니다. 필드가 우연히 겹치는 것(path, alt)과 의미적으로 같은 것은 다릅니다. Martin의 OCP 관점에서, 공통 부모 MediaBlock을 만들면 VideoBlock에만 필요한 변경이 MediaBlock을 수정하게 만들 수 있고, 그 수정이 ImageBlock/FeaturedImageBlock에 전파됩니다. 현재 코드베이스의 7개 Block이 모두 독립적인 flat hierarchy인 것은 의도적인 설계입니다.

(b) 새 primitive 추가 여부

추가하지 않습니다. editor.py docstring에 명시되어 있습니다: "upload_file covers images, videos, attachments. New block types never require new methods here." upload_file(path)는 Playwright의 file_choser를 통해 파일을 선택하는 primitive이며, .jpg든 .mp4든 동작이 동일합니다. SmartEditorOne은 수정 불필요합니다.

(c) 트랜스코딩 대기 위치

VideoStep.execute() 내부가 적합합니다. BlogEditor에 wait_for_transcode()를 추가하면 에디터가 "동영상 트랜스코딩"이라는 특정 블록의 세부사항을 알게 됩니다(SRP 위반). time.sleep(transcode_wait_ms/1000)은 가장 단순한 해결책이며, 테스트에서 transcode_wait_ms=0으로 설정하면 대기를 건너뛸 수 있습니다.

(d) media_id 해석 위치

VideoHandler가 해석하지 않습니다. 코드 흐름을 추적하면: LayoutSlot.config={"media_id":99} → job_builder._build_body_from_layout() → block_factory.create_block('video', config, media_resolver=resolver) → _resolve_media()가 media_id=99를 path='/uploads/99/intro.mp4'로 변환 → VideoBlock(path='/uploads/99/intro.mp4')이 생성됩니다. handler는 이미 해석된 path만 받습니다. 단, block_factory.py의 _MEDIA_BLOCK_TYPES에 'video'를 추가해야 합니다.

(e) 테스트 코드

```
def test_video_slot_with_media_id(self):
    resolver = lambda mid: f"/uploads/{mid}/intro.mp4"
    layout = _layout(
        _slot(0, "video", {"media_id": 99,
            "alt": "{keyword} intro", "autoplay": False}),
    )
    combo = _combo(keywords=[
        _keyword("region", "gangnam", cat_id=1),
        _keyword("subject", "math", cat_id=2)])
    blocks = _build_body_from_layout(
        layout, combo, media_resolver=resolver)[0].blocks
    assert isinstance(blocks[0], VideoBlock)
    assert blocks[0].path == "/uploads/99/intro.mp4"
    assert blocks[0].alt == "gangnam math intro"
    assert blocks[0].autoplay is False
```

(f) 기존 테스트 영향

_RecordingEditor: 수정 불필요. upload_file(path)를 이미 기록하므로 VideoStep도 처리됩니다. test_all_block_types_registered(): FACTORIES['video'] 등록을 빠뜨리면 실패합니다. validate_sections(): 수정 불필요. VideoBlock은 FeaturedImageBlock이 아니므로 "최대 1개" 규칙에 해당 없습니다.

문제 2. WordPress BlogEditor 구현

상황

시스템이 네이버 블로그 외에 WordPress를 지원해야 합니다. WordPress는 REST API(WP-JSON)로 포스트를 발행하며, 브라우저를 열 필요가 없습니다. Account.extra에 {"api_url": "https://...", "api_token": "..."} 형태로 API 정보가 저장됩니다. 이미지 업로드는 /wp-json/wp/v2/media 엔드포인트를 사용하고, 포스트 발행은 /wp-json/wp/v2/posts를 사용합니다.

현재 정적 구조 (변경 지점 표시)

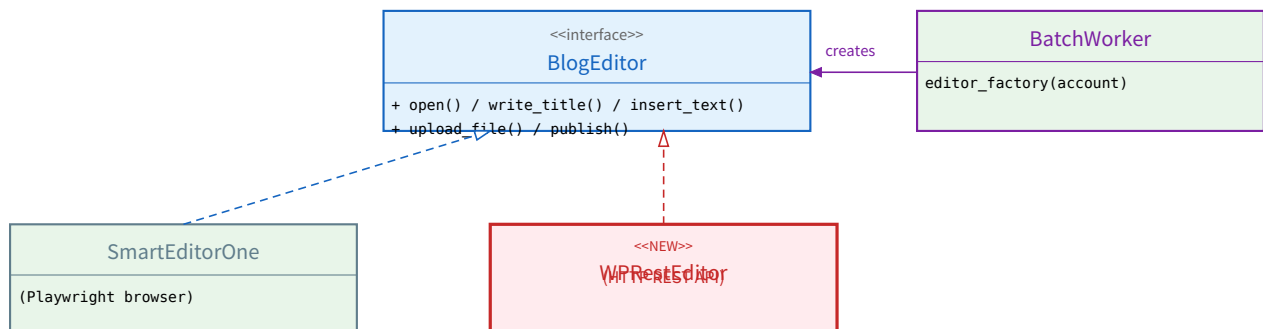


Fig 2-1. BlogEditor ABC — WPREstEditor가 새 구체 구현으로 추가

동적 흐름 (WPREstEditor 실행 시퀀스)

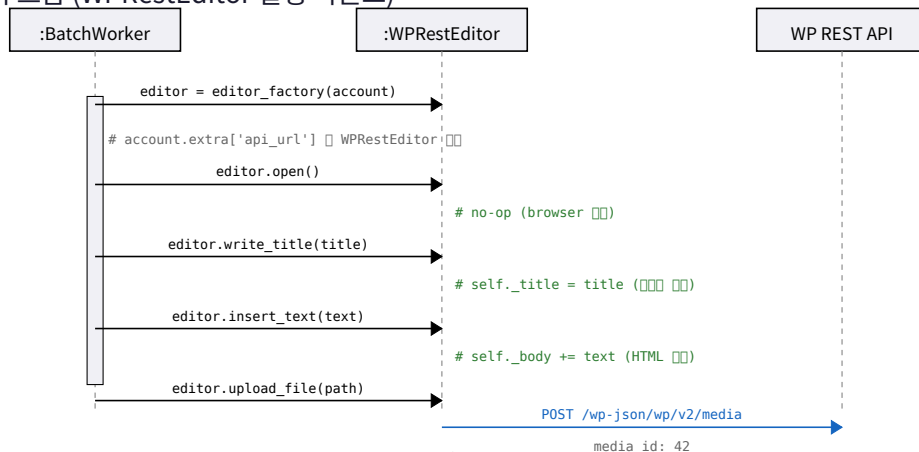


Fig 2-2. WPREstEditor 시퀀스 — BlogEditor 7 primitives를 REST API로 매핑

ERD 영향

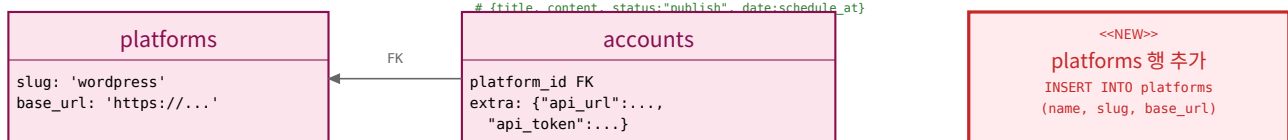


Fig 2-3. ERD — 새 테이블 없음, platforms에 행 추가, accounts.extra에 API 정보

질문

- (a) WPREstEditor 클래스를 작성하시오. BlogEditor의 7개 primitive 각각이 REST API의 어떤 동작에 매핑되는지 명시하시오. open()과 move_cursor()는 브라우저 기반이 아닌 환경에서 어떻게 처리하는가?
- (b) BatchWorker.editor_factory는 현재 Callable[[Any], BlogEditor] 타입입니다. 네이버와 WordPress를 어떻게 분기하는가? Account.platform.slug를 사용하는 방법과 별도 EditorRegistry를 만드는 방법의 OCP 관점 트레이드오프를 분석하시오.
- (c) SmartEditorOne은 Playwright Page 객체가 필요하지만 WPREstEditor는 HTTP client만 필요합니다. composition root(main.py)에서 editor_factory를 어떻게 구성하는가? 두 에디터의 생성자 시그니처가 다른 문제를 어떻게 해결하는가?
- (d) WPREstEditor.upload_file()이 반환하는 WordPress media_id를 어떻게 insert_text()의 HTML img 태그에 포함하는가? BlogEditor ABC가 upload_file()의 반환값을 void로 정의한 현재 설계의 한계를 분석하고, 최소 변경으로 해결하는 방법을 제시하시오.
- (e) test_runner.py의 _RecordingEditor와 동일한 패턴으로 WPREstEditor 전용 테스트 전략을 설계하시오. HTTP 호출을 어떻게 mock하는가? Port ABC 패턴과의 관계를 설명하시오.

해설

(a) WPRestEditor 매핑

```
class WPRestEditor(BlogEditor):
    def __init__(self, api_url, api_token):
        self._api = api_url
        self._headers = {"Authorization": f"Bearer {api_token}"}
        self._title = ""; self._body = ""; self._media_ids = []

    def open(self): pass # no-op: no browser
    def write_title(self, title): self._title = title
    def insert_text(self, text, newlines=2):
        self._body += text + "<br>" * newlines
    def upload_file(self, path):
        resp = requests.post(f"{self._api}/wp-json/wp/v2/media",
            headers=self._headers, files={"file": open(path, "rb")})
        self._media_ids.append(resp.json()["id"])
    def move_cursor(self, position="end"): pass # no-op
    def set_representative_media(self, index):
        self._featured = self._media_ids[index]
    def publish(self, schedule_at=None):
        payload = {"title": self._title, "content": self._body,
            "status": "publish", "featured_media": self._featured}
        if schedule_at:
            payload["status"] = "future"
            payload["date"] = schedule_at.isoformat()
        requests.post(f"{self._api}/wp-json/wp/v2/posts",
            headers=self._headers, json=payload)
```

open()과 move_cursor()는 no-op입니다. 이것은 LSP(리스코프 치환)를 위반하지 않습니다. BlogEditor ABC는 이 메서드들의 "구체적 동작"을 약속하지 않으며, "에디터를 사용할 준비를 하라"와 "커서를 이동하라"라는 의도만 약속합니다. REST API 기반 에디터에서 이 의도는 "아무것도 하지 않음"으로 충족됩니다.

(b) editor_factory 분기

방법 1: if/elif 분기 — editor_factory 내부에서 account.platform.slug를 보고 분기합니다. 단순하지만 새 플랫폼이 추가될 때마다 factory 함수를 수정해야 합니다(OCP 위반). 방법 2: EditorRegistry — {slug: constructor_fn} dict를 두고 slug로 lookup합니다. 새 플랫폼은 레지스트리에 등록만 하면 됩니다(OCP 준수). 이것은 HANDLERS dict, FACTORIES dict와 동일한 패턴이며, 코드베이스 내에 이미 확립된 관례입니다. 따라서 방법 2가 권장됩니다.

(c) 생성자 시그니처 차이 해결

SmartEditorOne(page, write_url)과 WPRestEditor(api_url, api_token)은 시그니처가 다릅니다. 해결: EditorRegistry에 등록하는 것은 생성자가 아니라 factory function입니다: registry['naver'] = lambda account: SmartEditorOne(create_page(account), get_write_url(account)), registry['wordpress'] = lambda account: WPRestEditor(account.extra['api_url'], account.extra['api_token']). BatchWorker.editor_factory의 시그니처 Callable[[Any], BlogEditor]는 그대로 유지됩니다.

(d) upload_file() 반환값 문제

현재 upload_file()은 void입니다. WPRestEditor는 내부 _media_ids 리스트에 저장하여 해결합니다. insert_text()가 호출될 때 마지막 media_id를 img 태그로 삽입합니다. BlogEditor ABC를 변경하지 않는 것이 핵심입니다. ABC를 수정하면 SmartEditorOne도 수정해야 하기 때문입니다. WPRestEditor 내부 상태로 해결하는 것이 최소 변경입니다.

(e) 테스트 전략

RecordingEditor 패턴을 적용합니다. WPRecordingEditor를 만들되, HTTP 호출 대신 호출 기록만 합니다. 혹은 WPRestEditor의 requests 호출을 responses 라이브러리로 mock합니다. 하지만 Port ABC 패턴을 따르려면 HTTP client 자체를 ABC로 추출하여 주입하는 것이 가장 깔끔합니다: class HttpClient(ABC): def post(self, url, **kw): ... 테스트에서 RecordingHttpClient를 주입합니다.

문제 3. 실패한 BatchItem 선택적 재시도

상황

Batch 40개 중 3개가 Gemini API 429(Rate Limit) 에러로 실패했습니다. 운영자가 실패한 3개만 골라 재시도하고 싶습니다. 재시도 시 같은 Account를 사용하되, 새 Batch로 묶어야 합니다. 재시도 횟수를 추적하고, 최대 3회까지만 허용해야 합니다. 원래 BatchItem과 재시도 BatchItem의 연결을 추적할 수 있어야 합니다.

ERD 변경 (새 컬럼 추가)

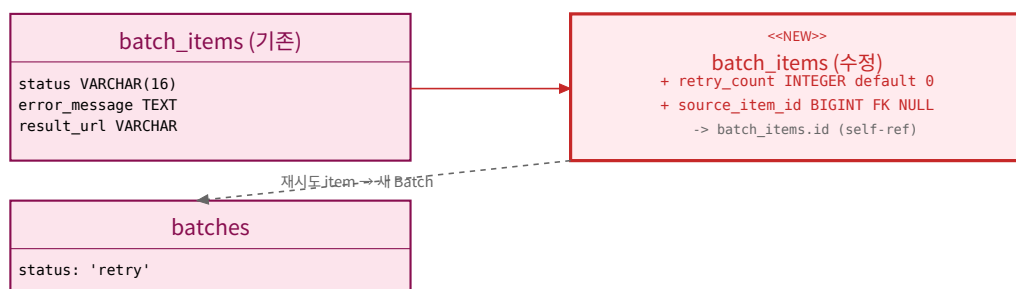


Fig 3-1. ERD — batch_items에 retry_count + source_item_id 컬럼 추가

동적 흐름 (재시도 시퀀스)

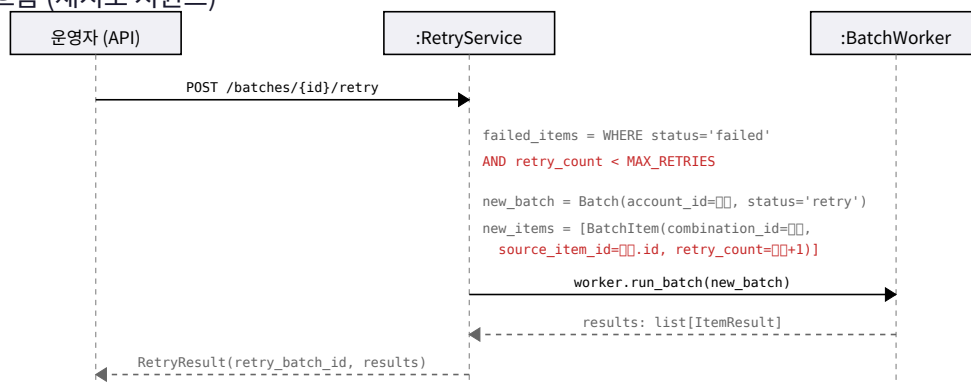


Fig 3-2. 재시도 시퀀스 — RetryService가 실패 item을 새 Batch로 묶어 재실행

질문

- (a) batch_items에 retry_count와 source_item_id를 추가하는 마이그레이션을 설계하시오. source_item_id는 자기 참조 FK입니다. 기존 데이터와의 하위 호환성을 어떻게 보장하는가?
- (b) RetryService를 설계하시오. 이 서비스는 factory 레이어에 속하는가, automator 레이어에 속하는가? PostingSpec과 JobRunner를 재사용하는가, 아니면 별도 파이프라인이 필요한가?
- (c) 재시도 시 Combination.used_at이 이미 설정되어 있습니다. BatchDispatcher._pending_combinations()가 used_at IS NULL을 조건으로 사용하므로, 재시도 조합이 '새로운 pending'으로 잘못 분류되지 않음을 어떻게 보장하는가?
- (d) 최대 재시도 3회 제한을 WHERE retry_count < 3으로 구현합니다. 이 상수를 어디에 정의해야 하는가? RunPreset.config에 넣는 방법과 시스템 상수로 두는 방법의 트레이드오프를 분석하시오.
- (e) source_item_id로 '원본 → 재시도1 → 재시도2' 체인을 추적하는 쿼리를 작성하시오. 이 체인이 순환 참조를 만들지 않음을 어떻게 보장하는가?

해설

(a) 마이그레이션 설계

ALTER TABLE batch_items ADD COLUMN retry_count INTEGER NOT NULL DEFAULT 0; ALTER TABLE batch_items ADD COLUMN source_item_id BIGINT NULL REFERENCES batch_items(id); 기존 행은 retry_count=0, source_item_id=NULL이므로 하위 호환됩니다. NOT NULL DEFAULT 0은 기존 데이터에 영향 없이 새 컬럼을 추가합니다.

(b) RetryService 위치

factory 레이어에 속합니다. RetryService는 batch/batch_items의 DB 상태를 관리합니다. 내부에서 build_posting_spec()과 runner.run()을 호출하므로 기존 파이프라인을 완전히 재사용합니다. 별도 파이프라인은 불필요합니다. BatchWorker.run_batch()를 그대로 호출하되, 새 Batch의 status를 'retry'로 표시하여 일반 배치와 구분합니다.

(c) 재시도와 pending 분류

Combination.used_at은 이미 설정되어 있으므로 _pending_combinations()에 잡히지 않습니다. 재시도는 Combination을 다시 pending으로 만드는 것이 아니라, 기존 BatchItem을 새 Batch에 복사하는 것입니다. new_item.combination_id = 원본.combination_id이므로 같은 조합을 참조하되, 새 BatchItem으로 실행합니다. Combination 테이블은 전혀 수정하지 않습니다.

(d) MAX_RETRIES 위치

RunPreset.config에 넣으면 캠페인별로 재시도 횟수를 제어할 수 있지만, RunSetting dataclass에 max_retries 필드를 추가해야 합니다. 시스템 상수로 두면 코드가 단순하지만 캠페인별 유연성이 없습니다. 현재 시스템에서 재시도는 예외적 상황이므로 시스템 상수 MAX_RETRIES=3이 적합합니다. 나중에 캠페인별 제어가 필요해지면 RunPreset으로 이전합니다(YAGNI 원칙).

(e) 체인 추적 쿼리

```
-- item_id=100의 모든 재시도
WITH RECURSIVE chain AS (
  SELECT id, source_item_id, retry_count, status
  FROM batch_items WHERE id = 100
  UNION ALL
  SELECT bi.id, bi.source_item_id, bi.retry_count, bi.status
  FROM batch_items bi JOIN chain c ON bi.source_item_id = c.id
)
SELECT * FROM chain ORDER BY retry_count;
```

순환 참조 방지: source_item_id는 항상 자신보다 작은 id를 참조합니다 (AUTO INCREMENT + 새 item은 항상 새 id). retry_count가 단조 증가하고 MAX_RETRIES로 상한이 있으므로 무한 체인은 불가능합니다.

문제 4. 캠페인 A/B 레이아웃 테스트

상황

운영자가 같은 캠페인에서 두 가지 레이아웃을 번갈아 사용하고 싶습니다. 레이아웃 A는 '제목 → 이미지 → 본문 3개', 레이아웃 B는 '제목 → 본문 2개 → 이미지 → CTA'입니다. Combination마다 어떤 레이아웃이 적용되었는지 추적해야 하며, 발행 후 각 레이아웃의 성과를 비교할 수 있어야 합니다. 현재 Campaign.layout_id는 단일 FK입니다.

ERD 변경

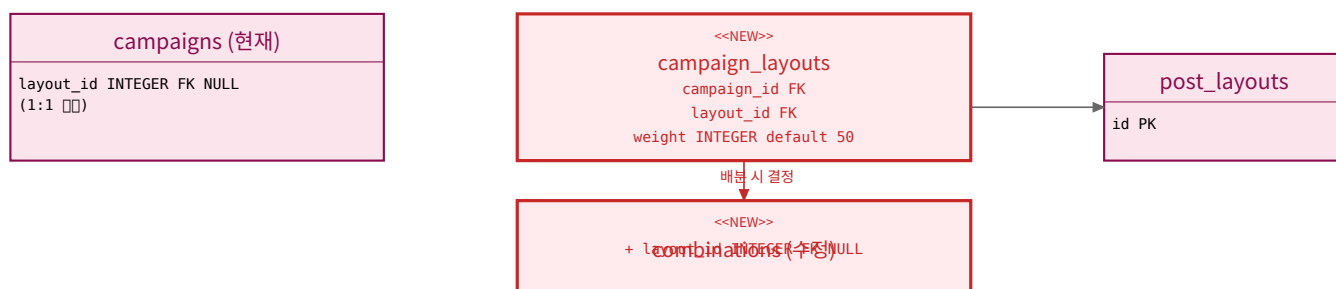


Fig 4-1. ERD — campaign_layouts 조인 테이블 추가 + combinations.layout_id 추가

정적 구조 변경

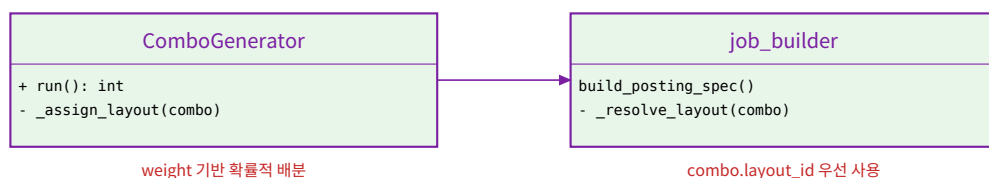


Fig 4-2. ComboGenerator가 layout을 배분, job_builder가 해석

질문

- (a) campaign_layouts 조인 테이블을 설계하시오. weight 필드가 확률적 배분에 어떻게 사용되는지 예시를 들어 설명하시오 (예: layout_A weight=70, layout_B weight=30이면?).
- (b) Campaign.layout_id(기존 1:1 FK)를 campaign_layouts(1:N)로 전환할 때, layout_id가 이미 설정된 기존 캠페인의 하위 호환성을 어떻게 보장하는가? 두 경로(layout_id, campaign_layouts)가 동시에 존재할 때의 우선순위를 설계하시오.
- (c) ComboGenerator.run()에서 layout을 배분하는 로직을 추가하시오. combination.layout_id에 선택된 layout의 id를 기록합니다. 이 배분은 순수 함수로 분리할 수 있는가? build_combos()와의 관계를 설명하시오.
- (d) job_builder.build_posting_spec()에서 combo.layout_id를 어떻게 사용하는가? 현재 campaign.layout을 사용하는 코드를 combo.layout_id 우선으로 수정하는 구체적인 코드 변경을 제시하시오.
- (e) A/B 성과 비교를 위해 batch_items.result_url과 combinations.layout_id를 JOIN하는 쿼리를 작성하시오. 레이아웃별 성공률을 계산하시오.

해설

(a) weight 기반 확률적 배분

campaign_layouts(campaign_id, layout_id, weight)에 (1, layout_A, 70), (1, layout_B, 30)이 등록되면 전체 weight 합은 100입니다. random.randint(1, 100)이 1-70이면 layout_A, 71-100이면 layout_B를 선택합니다. weight=50, 50이면 50:50 균등 배분입니다. weight는 비율이 아니라 상대 가중치이므로 (70, 30)과 (7, 3)은 동일합니다.

(b) 하위 호환성

우선순위: combinations.layout_id > campaign_layouts > campaign.layout_id. combo.layout_id가 있으면 그것을 사용(이미 배분됨). 없으면 campaign_layouts를 확인하여 weight 기반 선택. campaign_layouts도 없으면 campaign.layout_id 사용(기본 1:1). campaign.layout_id도 없으면 _build_body_default() 폴백. 기존 캠페인은 campaign_layouts가 비어 있으므로 자동으로 campaign.layout_id를 사용합니다.

(c) layout 배분의 순수 함수 분리

layout 선택은 랜덤이므로 완전한 순수 함수는 아닙니다. 하지만 random.Random(seed)를 주입하면 결정적으로 만들 수 있습니다: def pick_layout(weights: list[tuple[int, int]], rng: random.Random) -> int. 이것은 build_combos()와 별개입니다. build_combos()는 키워드 카르테시안 곱만 담당하고, layout 배분은 ComboGenerator._insert() 단계에서 각 Combination에 layout_id를 설정합니다.

(d) job_builder 수정

```
# job_builder.py - _build_body_from_layout
layout = None
if getattr(combo, "layout_id", None):
    layout = session.get(PostLayout, combo.layout_id) # combo
elif getattr(campaign, "layout", None):
    layout = campaign.layout # 1:1

body = (_build_body_from_layout(layout, combo, media_resolver)
        if layout else _build_body_default(combo))
```

(e) A/B 성과 비교 쿼리

```
SELECT c.layout_id, pl.name AS layout_name,
COUNT(*) AS total,
SUM(CASE WHEN bi.status = 'completed' THEN 1 ELSE 0 END) AS success,
ROUND(100.0 * SUM(CASE WHEN bi.status = 'completed'
THEN 1 ELSE 0 END) / COUNT(*), 1) AS success_rate
FROM batch_items bi
JOIN combinations c ON bi.combination_id = c.id
LEFT JOIN post_layouts pl ON c.layout_id = pl.id
WHERE c.campaign_id = 1
GROUP BY c.layout_id, pl.name;
```

문제 5. 계정 간 일일 포스팅 할당량

상황

플랫폼의 스팸 탐지가 강화되어 하나의 계정으로 하루에 최대 10개까지만 포스팅해야 합니다. 현재 `RunSetting.max_daily_posts=10`이 존재하지만 이것은 캠페인 단위이지 계정 단위가 아닙니다. 같은 계정이 캠페인 A에서 8개, 캠페인 B에서 5개를 포스팅하면 합계 13개로 한도를 초과합니다. `BatchDispatcher`가 계정의 '오늘 남은 할당량'을 고려하여 배분해야 합니다.

ERD 변경



Fig 5-1. ERD — daily_post_logs 테이블 추가 + accounts.daily_limit 컬럼 추가

동적 흐름 (Dispatcher 할당량 체크)

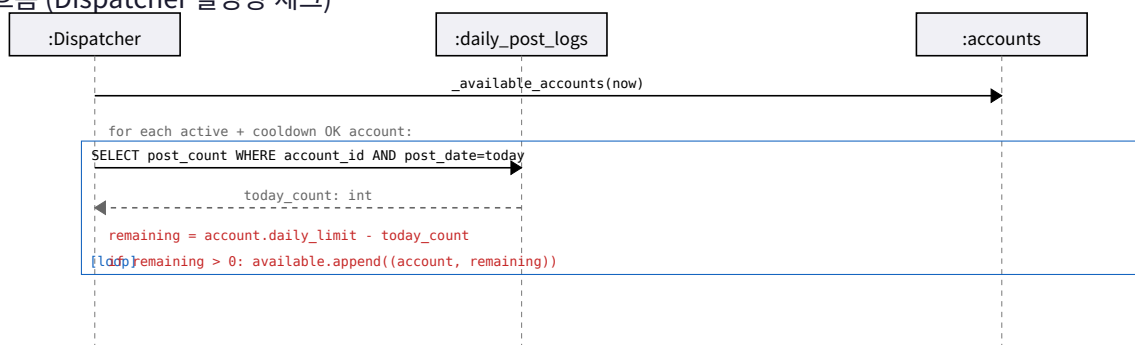


Fig 5-2. Dispatcher가 계정별 오늘 남은 할당량을 확인하는 시퀀스

질문

(a) `daily_post_logs` 테이블을 설계하시오. `UNIQUE(account_id, post_date)` 제약이 왜 필요한지, `UPSERT(INSERT ON CONFLICT UPDATE)` 패턴을 어떻게 활용하는지 설명하시오.

(b) `BatchDispatcher._available_accounts()`를 수정하시오. 현재는 `cooldown`만 확인하지만, '오늘 남은 할당량'도 확인해야 합니다. `remaining`이 0이면 계정을 사용하지 않고, `remaining`이 40(`BATCH_SIZE`) 미만이면 해당 계정에 `remaining`개만 배분해야 합니다. `_chunk()` 함수와의 상호작용을 설명하시오.

(c) `BatchWorker._process_item()`이 `item`을 성공적으로 처리한 후 `daily_post_logs`를 `UPDATE`해야 합니다. 이 로직을 `BatchWorker`에 넣는 것과 별도 `PostLogger` 서비스로 분리하는 것의 SRP 관점 트레이드오프를 분석하시오.

(d) 동시에 두 `BatchWorker`가 같은 계정으로 실행되면 할당량을 초과할 수 있습니다. 이 `race condition`을 방지하는 방법을 제시하시오. `worker_pid` 컬럼과의 관계를 설명하시오.

(e) 자정(KST 00:00)에 할당량이 리셋되어야 합니다. `daily_post_logs`를 삭제하는 방법과 새 날짜 행을 자동 생성하는 방법의 트레이드오프를 분석하시오. 시간대(KST) 처리의 주의점을 설명하시오.

해설

(a) daily_post_logs 설계

```
class DailyPostLog(Base):
    __tablename__ = "daily_post_logs"
    __table_args__ = (
        UniqueConstraint("account_id", "post_date",
            name="uq_account_date"),)
    id = mapped_column(Integer, primary_key=True)
    account_id = mapped_column(Integer,
        ForeignKey("accounts.id"), nullable=False)
    post_date = mapped_column(Date, nullable=False)
    post_count = mapped_column(Integer, nullable=False, default=0)
```

UNIQUE(account_id, post_date)는 '한 계정에 한 날짜에 하나의 로그 행'을 보장합니다. UPSERT 패턴: INSERT INTO daily_post_logs (account_id, post_date, post_count) VALUES (1, '2026-03-21', 1) ON CONFLICT (account_id, post_date) DO UPDATE SET post_count = post_count + 1. 이렇게 하면 첫 포스팅 시 행이 생성되고, 이후 포스팅 시 카운트만 증가합니다.

(b) Dispatcher 수정

현재 _available_accounts()는 (account,) 리스트를 반환합니다. 수정 후 (account, remaining) 튜플 리스트를 반환합니다. dispatch() 루프에서: chunk_size = min(BATCH_SIZE, remaining)으로 계정별 할당량을 존중합니다. remaining=25인 계정에는 최대 25개만 배분합니다. _chunk(combos, 40)으로 먼저 자르지 않고, 계정별 remaining으로 자릅니다: combos_for_this_account = combos[:remaining], combos = combos[remaining:].

(c) 로깅 위치

BatchWorker에 넣는 방법: _process_item() 성공 시 session.execute(UPSERT). 단순하지만 BatchWorker의 책임이 '배치 실행 + 할당량 기록' 두 가지가 됩니다(SRP 위반). PostLogger 분리: BatchWorker.run_batch() 후 호출자가 logger.log(results)를 호출. 깔끔하지만 호출자가 잊으면 로깅이 누락됩니다. 절충안: BatchWorker에 post_logger: PostLogger | None 을 주입하고, 성공 시 logger.log_post(account_id, date)를 호출합니다. None이면 로깅 안 함(기존 코드 하위 호환).

(d) Race condition 방지

현재 Batch.worker_pid는 '실행 중' 표시입니다. 하지만 할당량 초과 방지에는 부족합니다. 해결: BatchDispatcher.dispatch()에서 accounts를 SELECT FOR UPDATE로 잠급니다. dispatch()가 트랜잭션 안에서 실행되므로, 같은 계정을 두 dispatcher가 동시에 선택하는 것을 방지합니다. 또는 accounts에 assigned_today 컬럼을 두고 dispatch() 시점에 원자적으로 증가시킵니다.

(e) 자정 리셋

삭제 방법: 매일 DELETE FROM daily_post_logs WHERE post_date < today. 단순하지만 과거 이력이 사라집니다. 자동 생성 방법: 쿼리 시점에 WHERE post_date = today를 사용하면 해당 날짜의 행이 없으면 post_count=0과 동일합니다. 행을 삭제할 필요 없이 자연 리셋됩니다. KST 주의: post_date는 DATE 타입이므로 UTC가 아닌 KST 기준으로 today를 계산해야 합니다. datetime.now(tz=KST).date()를 사용합니다. UTC 23:30은 KST 08:30(다음 날)이므로 시간대 변환을 빠뜨리면 할당량이 잘못 계산됩니다.

이 문제집은 Martin의 원칙에 따라 "코드에서 다이어그램을 확인하라"를 실천합니다. 각 해설의 코드는 기존 테스트 코드의 패턴을 따르며, 기존 아키텍처를 최소한으로 변경하는 경로를 제시합니다.